TABLE OF CONTENT

S.NO.	TITLE	PAGE NO.
1.	Introduction	2
2.	Rationale	2
3.	Objectives	3
4.	Feasibility Study	3
5.	Methodology/ Planning of Work	4
6.	Software Required for the Project's Development	5
7.	Expected Outcomes	6

1. Introduction

- **Multi Agent Student Productivity Assistant** is a multi-agent student productivity assistant that automates routine academic and administrative tasks through a network of cooperating AI agents rather than a single monolithic chatbot.
- It functions as a **personal executive assistant** for students, handling communication drafting, scheduling, and progress tracking on their behalf.
- It targets students who want to **reduce time spent on repetitive tasks** such as writing formal emails and manually logging study hours.
- A key feature is the **HR/Faculty Email Agent**, which converts a single natural-language command into a fully drafted, formatted email, refined through a **Human-in-the-Loop (HITL)** review cycle before it is scheduled or sent.
- It includes a **Voice-Based Study Scheduler** that lets users create and manage study blocks simply by speaking, using speech-to-text and natural-language date/time parsing.
- A **Study Tracking Agent** follows up on every scheduled study block, checks in with the user, and logs actual hours studied against the planned schedule.
- The system is built as a **graph-based multi-agent workflow** using LangGraph, where each agent (email, scheduler, tracker) is a node sharing a common state object.
- The architecture is modular, so new agents (e.g., a reminder agent or a report-generation agent) can be added as additional graph nodes without redesigning the system.
- The use of **LLM-powered agents with tool access** fills a gap between generic to-do apps and true task-executing assistants for students.
- The project tech stack includes **Python and LangGraph** for the agentic backend, **Google Gemini** (with Groq as an alternate LLM provider) for language understanding and generation, and **Flutter** for the cross-platform mobile front end.
- It demonstrates the real-world potential of combining agentic AI workflows, human oversight, and mobile app engineering into a single productivity tool.

2. Rationale

Students routinely lose valuable time on small but effortful administrative tasks: composing formally worded emails to HR departments, faculty, or recruiters; manually creating study timetables; and tracking whether those study plans were actually followed. Existing productivity apps are largely passive record-keepers they store what the user types but do not act on the user's behalf, and they cannot verify whether a planned task was actually completed. Multi Agent Student Productivity Assistant is motivated by the need for an assistant that is **proactive and agentic**: one that drafts communication, listens to spoken

commands, and actively follows up on commitments, rather than simply displaying a calendar or a notes list.

The project also demonstrates how a **graph-based multi-agent architecture** built with LangGraph can coordinate several specialised LLM-driven agents around a shared state, while a **Human-in-the-Loop checkpoint** keeps the user in control of any outgoing communication. By combining a free-tier LLM (Gemini), voice interaction, and a Flutter mobile interface, Multi Agent Student Productivity Assistant shows how modern agentic AI engineering can turn tedious student workflows into a fast, conversational experience, while remaining accessible and cost-free to build and run.

3. Objectives

- To design and implement a **multi-agent architecture using LangGraph** in which specialised agents for email drafting, scheduling, and study tracking cooperate through a shared, graph-based state.
- To build an **HR/Faculty email-generation agent with Human-in-the-Loop approval**, where the agent iteratively regenerates the email draft based on user feedback until the user approves it for scheduling/sending.
- To enable **voice-based creation of study blocks**, converting spoken commands into structured, schedulable study sessions using speech-to-text and LLM-based intent/date-time extraction.
- To develop a **study-hours tracking agent** that follows up on each scheduled study block, prompts the user for confirmation, and logs actual study hours against the plan.
- To provide a **Flutter-based mobile interface** that exposes all agent capabilities (email review, voice scheduling, progress tracking) through a simple, unified student-facing app.

4. Feasibility Study

Feasibility:

Multi Agent Student Productivity Assistant is highly feasible because it is built on mature, well-documented, and largely free-to-use technologies: LangGraph for orchestrating agent workflows, Google Gemini's free-tier API (with Groq as a fast, low-cost fallback) for LLM inference, and Flutter for a single cross-platform mobile codebase. LangGraph's graph-based execution model maps naturally onto the project's needs conditional loops for the HITL email-review cycle, and branching nodes for the scheduler and tracker agents making the system straightforward to build incrementally, test agent-by-agent, and extend later.

Need:

There is a genuine need for an assistant like this because students routinely delay or avoid sending formal emails due to the effort of getting the wording right, and they frequently create study schedules that are never followed up on or verified. Generic calendar and to-do apps can store a plan but cannot draft

communication or check whether the plan was executed. Multi Agent Student Productivity Assistant addresses this gap by pairing an LLM-driven drafting-and-review loop for emails with an active follow-up agent for study commitments, giving students a single assistant that both creates and enforces their plans.

Significance:

Multi Agent Student Productivity Assistant is significant because it demonstrates how a graph-based multi-agent system, combined with a human-in-the-loop safety checkpoint, can safely automate a task (sending an email) that most users are hesitant to fully delegate. By adding voice-driven scheduling and automated progress tracking, the project also shows how conversational, agentic AI can move beyond passive note-taking apps toward assistants that actively manage a user's commitments. Socially, the project promotes better time-management habits and reduces the communication overhead that often discourages students from reaching out to HR, faculty, or recruiters.

5. Methodology/ Planning of Work

Research Type:

The project will follow an **applied, design-and-development research approach**, focused on building and evaluating an agentic AI system for real student workflows email communication, study scheduling, and progress tracking.

Research Unit:

The unit of analysis is the **individual student user session**: each interaction cycle (a single command, a voice input, or a scheduled study block and its follow-up) forms one traceable unit of agent activity within the LangGraph state.

Data Collection Methods:

- **User Commands & Voice Input:** Natural-language text/voice commands provided by the student (e.g., "schedule a 2-hour study block for DBMS tomorrow at 5 PM").
- **HITL Feedback Data:** User approval/rejection feedback on each generated email draft, used to refine subsequent regenerations within the same session.
- **Study Session Logs:** Timestamps of scheduled vs. actually completed study blocks, captured through the tracking agent's follow-up prompts.
- **System & Agent Logs:** LangGraph execution traces (node transitions, tool calls, state snapshots) used to debug and improve agent behaviour.

Tools for Development & Analysis:

- **LangGraph** — for defining the multi-agent graph: nodes for the Email Agent, Scheduler Agent, and Tracking Agent, with conditional edges for the HITL loop.

- **Google Gemini API / Groq API** — as the LLM backbone for natural-language understanding, email drafting, and intent extraction from voice input.
- **Speech-to-Text (e.g., Google Speech-to-Text / on-device Flutter STT)** — to convert spoken study-block commands into text for the agent pipeline.
- **Python (FastAPI)** — to expose the LangGraph agent pipeline as an API consumed by the Flutter app.
- **Flutter & Dart** — for building the cross-platform mobile front end and voice-input UI.
- **SQLite / Firebase** — for persisting schedules, study logs, and email-draft history between sessions.

Project Timeline / Milestones:

The project is planned to be completed within a period of one month (4 weeks), with each week dedicated to a specific set of deliverables as outlined below:

Week	Duration	Work Planned
Week 1	Days 1–7	Requirement analysis and finalisation of scope; set up Python, LangGraph, and Gemini/Groq API access; design the overall multi-agent architecture and the shared LangGraph state schema; create the system/agent-flow diagram.
Week 2	Days 8–14	Develop the HR/Faculty Email Agent: prompt design for email drafting, LangGraph node for iterative regeneration, and the Human-in-the-Loop approval/feedback loop; build the corresponding Flutter screen for reviewing and approving drafts.
Week 3	Days 15–21	Develop the Voice-Based Study Scheduler: integrate speech-to-text, build LLM-based intent and date/time extraction, and implement the LangGraph scheduling node; build the Flutter voice-input and schedule-management UI; begin designing the Study Tracking Agent.
Week 4	Days 22–30	Implement the Study Tracking Agent (follow-up prompts and hours logging); integrate all three agents into a single LangGraph multi-agent graph; connect the Flutter app to the backend API end-to-end; perform integration testing, bug fixing, and prepare final documentation and demo.

6. Software/Hardware Required for the Project's Development

Software Requirements:

- **Programming Languages:**
    - **Python 3.11+:** Backend language used for building the LangGraph agent pipeline and API layer.

- **Dart / Flutter:** Flutter is an open-source UI software development kit created by Google, used here to build a single cross-platform mobile app for Android and iOS.

- **Agent Framework & LLM:**
  - **LangGraph:** Framework for building and orchestrating the stateful, graph-based multi-agent workflow.
  - **Google Gemini API (primary) / Groq API (alternate):** LLM providers used for language understanding, email generation, and intent extraction, chosen for their free/low-cost access tiers.
- **Backend Services & Communication:**
  - **FastAPI:** Used to expose the LangGraph agent pipeline as REST/WebSocket endpoints consumed by the Flutter app.
- **Database:**
  - Firebase / SQLite: Storage for user schedules, study logs, email drafts, and agent state.

Hardware Requirements:

- **Development Machine:**
  - Laptop/PC with at least 8GB RAM: Required for running the backend services, LangGraph pipeline, and Flutter build tools.
  - Processor: Intel i5 or equivalent for standard development and testing tasks.
- **Mobile Test Device:**
  - An Android/iOS smartphone or emulator for testing voice input, notifications, and the Flutter UI.
- **Network Access:**
  - Stable internet connection: Required for real-time LLM API calls (Gemini/Groq) and cloud data sync.

7. Expected Outcomes

- A fully functional multi-agent assistant that drafts, refines, and schedules emails to HR/faculty through a Human-in-the-Loop approval workflow.
- A voice-based study-block scheduler that accurately converts spoken commands into structured, saved study sessions.
- An active Study Tracking Agent that follows up on scheduled study blocks and logs actual hours studied against the plan.

- A scalable, graph-based multi-agent backend built with LangGraph, allowing new agents to be added with minimal changes to existing workflows.
- A cross-platform Flutter mobile application providing a single, unified interface to all agent capabilities.
- Successful demonstration of combining agentic AI, human-in-the-loop safety, voice interaction, and mobile engineering into one cohesive student productivity tool.

Signature of Student 1–

Signature of Student 2–

Signature of Student 3–